Actividad 5

Contador

Índice

1. Introducción y objetivos 2
2. Creación del proyecto 4
3. Descripción VHDL del biestable tipo D 4
4. Descripción VHDL del contador 7
5. Simulación del contador 10
6. Asignación de pines de la FPGA 12
7. Estructura interna de los CLB 14
8. Configuración del contador en la FPGA 18
9. Recursos de la FPGA empleados 19
Referencias 21

Actividad 5

Contador

1.- Introducción y objetivos

En esta práctica se va a proceder a implementar en la FPGA de la tarjeta de desarrollo un circuito secuencial, en este caso, un contador.

El contador binario es uno de los circuitos secuenciales más básicos y más empleados en diseño digital. En nuestro caso, vamos a diseñar un contador binario de 4 bits, que recorra la secuencia de 0 a 9 de manera cíclica (este circuito se llama contador módulo 10), es decir, los números *BCD*. Este contador dispondrá de una señal reloj, *clk*, y señales de control *reset* y *ce*, según la Figura 1. La salida, puerto *count*, debe ser de 4 bits, que son los necesarios para poder representar en binario puro el número más grande del contador, en este caso 9 (1001).

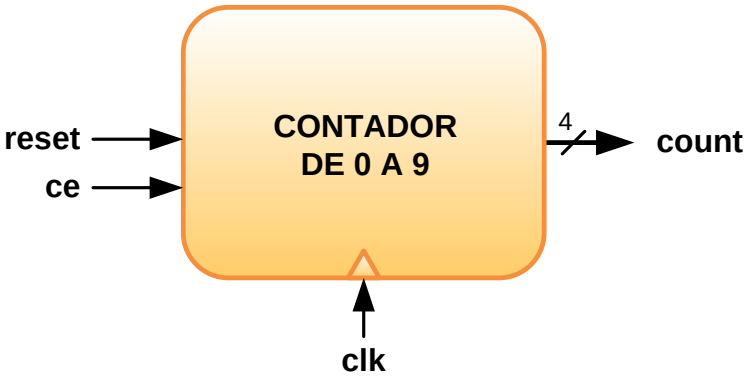

Figura 1. Esquema del contador binario de 4 bits módulo 10.

El contador a implementar se especifica a través de la siguiente tabla de transiciones:

QA	QB	QC	QD	QA^+	QB^+	QC^+	QD^+
0	0	0	0	0	0	0	1
0	0	0	1	0	0	1	0
0	0	1	0	0	0	1	1
0	0	1	1	0	1	0	0
0	1	0	0	0	1	0	1
0	1	0	1	0	1	1	0
0	1	1	0	0	1	1	1
0	1	1	1	1	0	0	0
1	0	0	0	1	0	0	1

1	0	0	1	0	0	0	0
1	0	1	0	X	X	X	X
1	0	1	1	X	X	X	X
1	1	0	0	X	X	X	X
1	1	0	1	X	X	X	X
1	1	1	0	X	X	X	X
1	1	1	1	X	X	X	X

Tabla 1. Tabla de transiciones del contador de 0 a 9.

Como se aprecia, son necesarios 4 flip-flops para implementar el contador. Por otra parte, el estado siguiente de los flip-flops tipo D se rige por esta ecuación: $Q^+ = D$. Así, las entradas a los 4 biestables se obtienen directamente de la parte derecha de la tabla. Simplificando, las funciones lógicas que deben ingresar por las entradas a los biestables se muestran en la Ecuación 1.

$$
\begin{aligned}
D_A &= Q_A\overline{Q_D} + Q_BQ_CQ_D \\
D_B &= Q_B\overline{Q_C} + Q_B\overline{Q_D} + \overline{Q_B}Q_CQ_D = Q_B\left(\overline{Q_C} + \overline{Q_D}\right) + \overline{Q_B}Q_CQ_D \\
D_C &= \overline{Q_A}\cdot\overline{Q_C}Q_D + \overline{Q_A}Q_C\overline{Q_D} = \overline{Q_A}(Q_C \oplus Q_D) \\
D_D &= \overline{Q_D}
\end{aligned}
$$

Ecuación 1. Ecuaciones algebraicas de la entrada a cada flip-flop.

Con esto, el circuito que debemos especificar en VHDL es el que se muestra en la Figura 2.

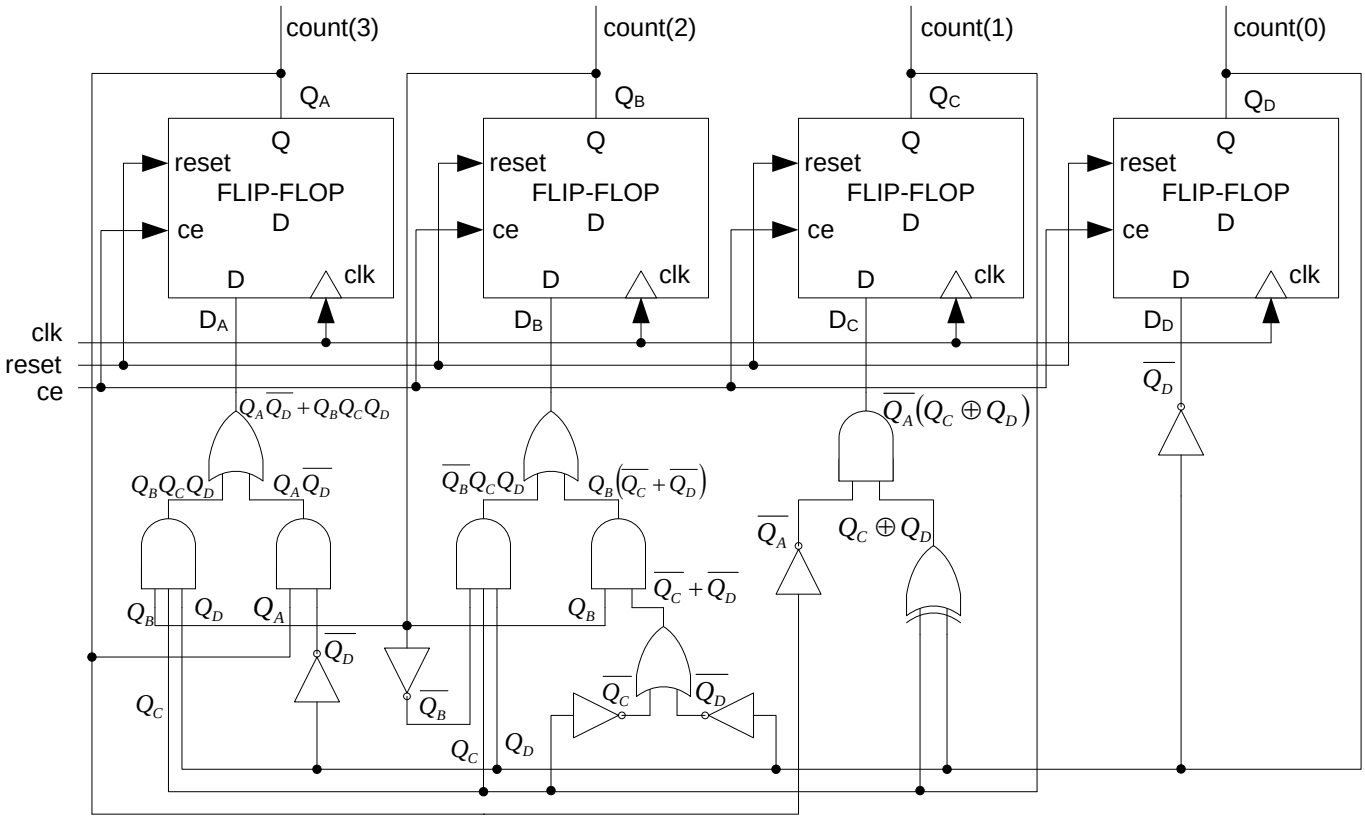

Figura 2. Circuito digital del contador de 0 a 9.

Los objetivos a alcanzar en esta práctica son:

- Conocer los procesos implícitos y explícitos
- Describir un flip-flop tipo D en VHDL
- Diseño jerárquico
- Profundizar en la arquitectura interna de un CLB
- Analizar los recursos empleados en el diseño
- Implementación en la tarjeta de desarrollo contador módulo 10

2.- Creación del proyecto

Para la creación del proyecto de Vivado de esta actividad, deben seguirse los pasos descritos en el Apartado 3 de la Actividad 0. En este caso, el nombre del proyecto es *Actividad5* (todo junto) y el resto de parámetros se deja igual.

3.- Descripción VHDL del biestable tipo D

Para implementar el contador de 0 a 9 son necesarios 4 biestables tipo D (Figura 2). Así pues, el primer paso en el proyecto consiste en crear un código VHDL que infiera dicho biestable. La descripción de este componente debe contemplar un puerto de habilitación o *chip enable* (ce) y un puerto de *reset* (Figura 3).

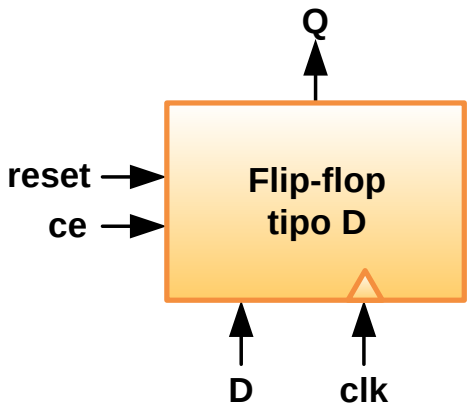

Figura 3. Esquema de un flip-flop tipo D.

Para generar un código VHDL que infiera el uso de un flip-flop tipo D en la FPGA se procede así:

1. Generar la plantilla del código VHDL del biestable, *ffD_reset.vhd*, a través del comando *New Source*, siguiendo los pasos descritos en el apartado 3 de la Actividad 1, añadiendo en el asistente los puertos del componente (Figura 4).

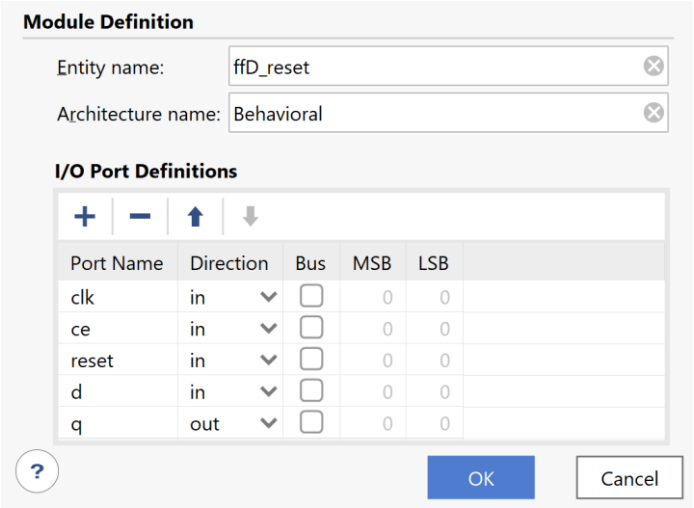

Figura 4. Definición de los puertos del biestable tipo D.

2. En la plantilla generada, escribir el código en la arquitectura (Figura 5).

La funcionalidad del biestable está descrita en el proceso (líneas 16-25). Sin entrar en pormenores, las ecuaciones booleanas que se implementaron en las prácticas anteriores son **procesos implícitos**, aunque también pueden expresarse según la sintaxis de la Figura 5 (**procesos explícitos**, porque aparece expresamente la palabra *process*). Respecto al proceso de este código cabe destacar:

- En la línea 16 aparece la denominada lista de sensibilidad del proceso. Éste no se ejecutará a menos que haya un cambio en alguna de las señales presentes en esta lista. En este caso, la única asignación del proceso, a la señal *q*, sólo va a cambiar en que caso que de que cambie la única señal de la lista de sensibilidad del proceso, es decir, *clk*.
- En la línea 18 se encuentra una condición que se cumple cuando se produce un evento en la señal *clk* (con *clk'event*), es decir, cuando esta señal pasa de 0 a 1 ó de 1 a 0; y además que clk valga 1. Esto es equivalente a decir que la condición se cumple cuando se produce un flanco de subida en la señal de reloj *clk* (esto es, la transición de 0 a 1).

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity ffD_reset is
5      Port ( clk : in STD_LOGIC;
6             ce : in STD_LOGIC;
7             reset : in STD_LOGIC;
8             d : in STD_LOGIC;
9             q : out STD_LOGIC);
10 end ffD_reset;
11
12 architecture Behavioral of ffD_reset is
13
14 begin
15
16 process (clk)
17 begin
18    if clk'event and clk='1' then
19       if reset='1' then
20          q <= '0';
21       elsif ce ='1' then
22          q <= d;
23       end if;
24    end if;
25 end process;
26
27 end Behavioral;
```

Figura 5. Código VHDL del biestable tipo D.

- En la segunda condición, anidada a la primera, se especifica que, si se produce un *reset*, la salida *q* debe ser 0 y, si no, el valor de entrada, *d*, si está habilitado (ce a 1). Todo esto sólo si se produce un flanco de subida en la señal de reloj.

Los procesos explícitos son muy importantes, ya que podemos describir un sistema digital de una manera algorítmica, facilitando la vida a los ingenieros electrónicos y reduciendo considerablemente el tiempo de diseño. El sintetizador se encargará de traducir esta descripción en el hardware oportuno, en este, caso, un flip-flop tipo D, activo a flanco de subida, con habilitación, y *reset* síncrono activo a nivel alta.

Para ilustrar un ejemplo de un proceso explícito de un circuito combinacional, se muestra la Figura 6. Nótese la diferencia respecto a la descripción del mismo circuito con un proceso implícito (línea 22 de la Figura 11 de la Actividad 1).

```
process (a,b)
begin
    z <= a and b;
end process;
```

Figura 6. Descripción de una puerta *and* con un proceso explícito.

4.- Descripción VHDL del contador

Una vez creado el módulo del flip-flop, el siguiente paso consiste en generar el contador, en el que se deben usar 4 flip-flops como subcircuitos. Para generar el módulo *contador.vhd* se procede así:

1. Generar la plantilla del código VHDL del biestable, *contador.vhd*, a través del comando *New Source*, siguiendo los pasos descritos en el apartado 3 de la Actividad 1, añadiendo en el asistente los puertos del componente (Figura 7).

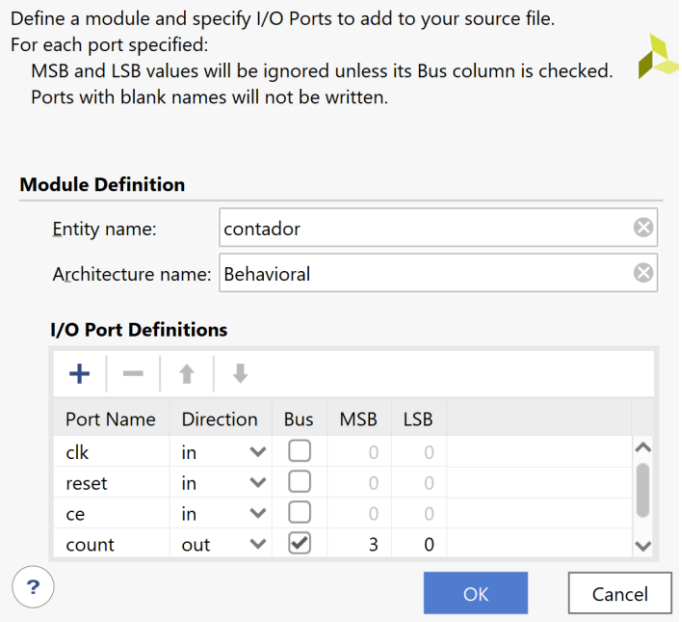

Figura 7. Definición de los puertos del contador.

2. En la plantilla generada, escribir el código en la arquitectura (Figura 8).

En la parte declarativa de la arquitectura debe aparecer la declaración del subcircuito que se a usar, a través de la cláusula COMPONENT, así como las señales internas del contador que se van a usar para conectar cada una de las entradas y salidas de datos de los biestables (*qa*, *qb*, *qc*, *qd*, *da*, *db*, *dc* y *dd*). Esta sección del código se muestra enmarcada en rojo en la Figura 8.

En la parte funcional de la arquitectura debe describirse el circuito del contador de acuerdo con la Figura 2. Para ello, se debe hacer uso de 4 biestables del tipo declarado, haciendo uso de la cláusula PORT MAP. A la hora de realizar el mapeo de puertos, se puede apreciar que los 4 biestables tienen en común las entradas *clk*, *reset* y *ce*. Las entradas *d* y las salidas *q* de cada biestable son conectadas a las señales internas *qa*, *qb*, *qc*, *qd*, *da*, *db*, *dc* y *dd* previamente declaradas (rectángulo verde en la Figura 8).

```
11 architecture Behavioral of contador is
12
13     component ffD_reset
14         port (clk   : in std_logic;
15               ce    : in std_logic;
16               reset : in std_logic;
17               d     : in std_logic;
18               q     : out std_logic);
19     end component;
20     signal qa, qb, qc, qd: std_logic;
21     signal da, db, dc, dd: std_logic;
22
23 begin
24
25   unitA : ffD_reset
26     port map (clk   => clk,
27               ce    => ce,
28               reset => reset,
29               d     => da,
30               q     => qa);
31   unitB : ffD_reset
32     port map (clk   => clk,
33               ce    => ce,
34               reset => reset,
35               d     => db,
36               q     => qb);
37   unitC : ffD_reset
38     port map (clk   => clk,
39               ce    => ce,
40               reset => reset,
41               d     => dc,
42               q     => qc);
43   unitD : ffD_reset
44     port map (clk   => clk,
45               ce    => ce,
46               reset => reset,
47               d     => dd,
48               q     => qd);
49
50     dd <= not qd;
51     dc <= not qa and (qc xor qd);
52     db <= (not qb and qc and qd) or (qb and (not qc or not qd));
53     da <= (qa and not qd) or (qb and qc and qd);
54
55     count <= qa & qb & qc & qd; --concatenación
56 --     count(0) <= qd;
57 --     count(1) <= qc;
58 --     count(2) <= qb;
59 --     count(3) <= qa;
60
61 end Behavioral;
```

Figura 8. Código VHDL de la arquitectura de *contador.vhd*.

Por último, queda establecer la lógica de entrada a cada biestable, según la Ecuación 1, además de conectar la salida de cada biestable al puerto de salida de 4 bits *count*. Esta lógica y sus conexiones se especifican en el rectángulo naranja de la Figura 8. En la línea 55 se ha empleado el operador concatenación (&) para realizar la conexión con el puerto de salida. Otra manera de realizar la conexión es con asignaciones (comentado en las líneas 56-59). Es importante tener en cuenta que en las asignaciones de la Ecuación 1 no se puede usar directamente el puerto *count*, porque éste está declarado como puerto de salida y, por tanto, no puede estar a la derecha de las asignaciones (confrontar con el Apartado 5 de la Actividad 1). Es por esto que se ha tenido que declarar las señales internas *qa*, *qb*, *qc* y *qd*.

En la Figura 9, se muestra el circuito que se describe en *contador.vhd* donde están resaltados los componentes asociados con el color correspondiente a cada sección del código de la Figura 8.

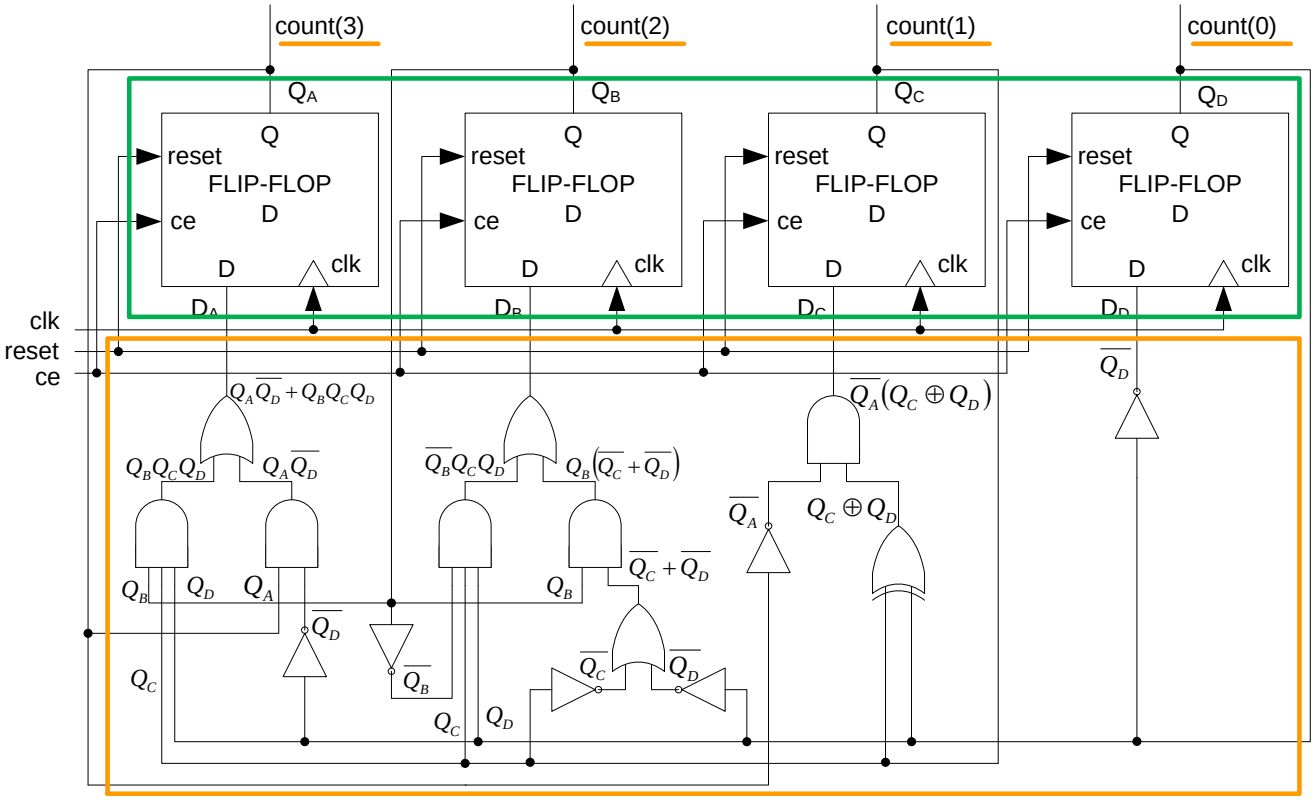

Figura 9. Circuito digital del contador de 0 a 9 descrita en *contador.vhd*.

Una vez realizado el circuito del contador, la ventana de jerarquía debe mostrar los 4 subcircuitos *ffD_reset* usados colgando del circuito *contador*, como puede apreciarse en la Figura 10.

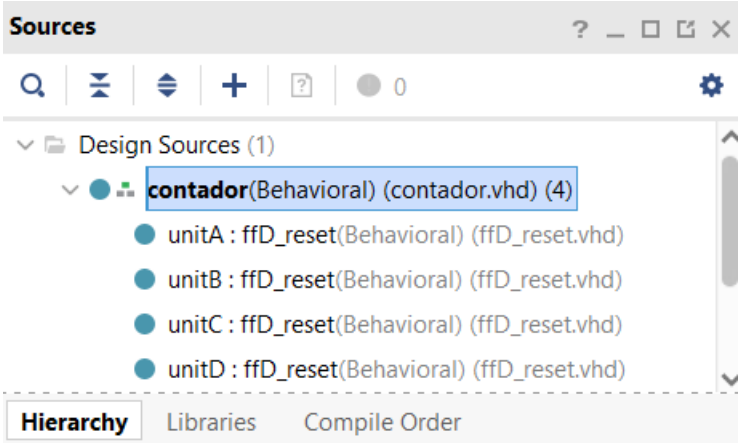

Figura 10. Captura de pantalla de la pestaña de jerarquía de *contador*.

Este es un ejemplo muy básico de un diseño jerárquico. En diseños complejos, lo normal es particionar un circuito muy complicado y descomponerlo en pequeñas partes que son fáciles de describir e implementar. A continuación, se van juntando estos subcircuitos como si piezas de un puzle se tratara. Es lo que se denomina una metodología de diseño *bottom-top*.

5.- Simulación del contador

La simulación del contador requiere la creación del *test bench* correspondiente. Para la creación *test bench*, deben seguirse los pasos descritos en el Apartado 6 de la Actividad 1. En este caso, el nombre del *test bench* se puede poner *contador_tb*. La arquitectura se muestra en la Figura 11.

Respecto al código de la Figura 11 se puede realizar las siguientes consideraciones:

- Las señales internas declaradas están inicializadas a 0 (líneas 17 a 19).
- En este caso, el *test bench* resulta más simplificado, ya que la señal de reloj, que se genera en la línea 31, oscila indefinidamente hasta el final del tiempo de simulación. No hay, como en los otros *test bench* de las actividades anteriores, ninguna condición en la que finalice la asignación.
- En el proceso de la línea 33 se modifican las señales de control para poder comprobar el correcto funcionamiento del contador en distintos estados. En este caso, el circuito está reseteado y

deshabilitado los primeros 100 ns. A continuación, se deja de resetear, y, posteriormente, tras otros 100 ns, se habilita para que se inicie la cuenta. Los tiempos que se han usado son arbitrarios.

```
8  architecture Behavioral of contador_tb is
9
10     component contador
11         port (clk   : in std_logic;
12               reset : in std_logic;
13               ce    : in std_logic;
14               count : out std_logic_vector (3 downto 0));
15     end component;
16
17     signal clk   : std_logic := '0';
18     signal reset : std_logic := '0';
19     signal ce    : std_logic := '0';
20     signal count : std_logic_vector (3 downto 0);
21
22 begin
23
24     dut : contador
25     port map (clk   => clk,
26               reset => reset,
27               ce    => ce,
28               count => count);
29
30     -- Clock generation
31     clk <= not clk after 10 ns;
32
33     stimuli : process
34     begin
35         -- EDIT Adapt initialization as needed
36         ce <= '0'; reset <= '1';
37         wait for 100 ns;
38         reset <= '0';
39         wait for 100 ns;
40         ce <= '1';
41         wait;
42     end process;
43
44 end Behavioral;
```

Figura 11. Captura de pantalla de la arquitectura de *contador_tb.vhd*.

La Figura 12 muestra el cronograma de simulación generado con el *test bench*. Se puede apreciar que el circuito se comporta adecuadamente, es decir, cuenta de 0 a 9 cíclicamente. Una vez llegado al 9, vuelve a 0, y así,

indefinidamente mientras esté habilitado el contador y no se efectúe un reset.

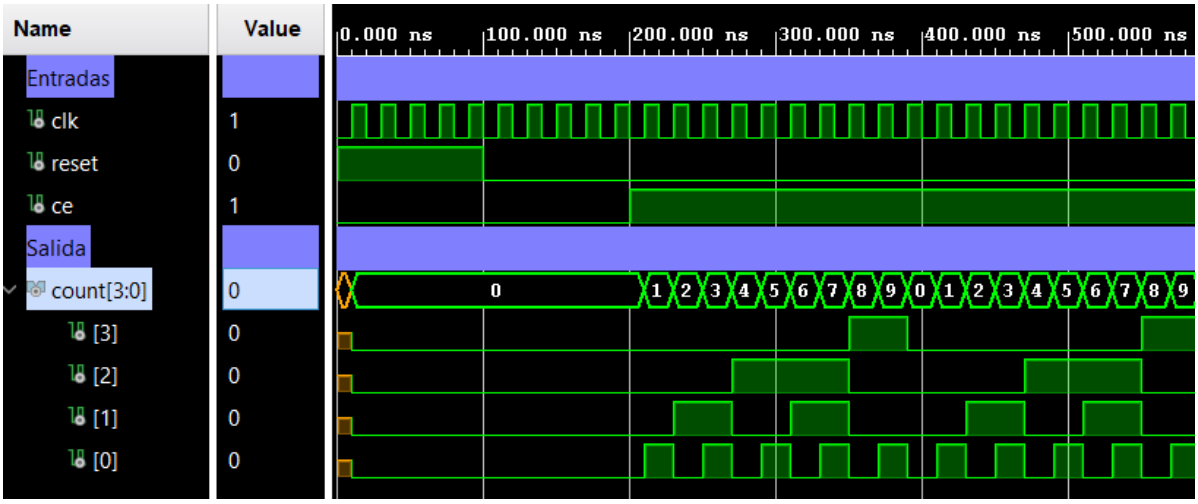

Figura 12. Captura de pantalla del cronograma de simulación del contador.

6.- Asignación de pines de la FPGA

Para asignar qué pines de la FPGA se van a conectar a los puertos de entrada y salida del contador binario de 0 a 9 hay que seguir el procedimiento especificado del Apartado 3 y siguientes de la Actividad 3. En este caso, las entradas de *reset* y *ce* se van a conectar a los pines de la FPGA que están conectados a los 2 interruptores más a la derecha de la tarjeta, la entrada de reloj se va a conectar al botón central de la tarjeta (de modo que fácilmente se puede generar una señal de reloj manual pulsando en él reiteradamente); y las salidas, a los pines de la FPGA conectados a los 4 leds más a la derecha del array.

Así pues, de los periféricos de la tarjeta se usan 2 interruptores, un botón y 4 leds. El archivo de restricciones de usuario que queda usando la plantilla maestra se muestra en la Figura 12.

Como novedad, en este caso es necesario incluir la directiva de la línea 9 para la señal de reloj *clk*. Esta directiva es necesaria para poder usar el pin de la FPGA N17 (línea 77) como señal de reloj y no una línea dedicada y diseñada para conducir este tipo de señales (como el pin E3, ver línea 7, que está comentada). Si no introducimos esta directiva, a la hora de realizar la implementación saltará un error y no se podrá generar el archivo de configuración.

```
6  ## Clock signal
7  #set_property -dict { PACKAGE_PIN E3    IOSTANDARD LVCMOS33 } [get_ports { CLK100MHZ }];
8  #create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports {CLK100MHZ}];
9  set_property CLOCK_DEDICATED_ROUTE FALSE [get_nets clk_IBUF]
10
11 ##Switches
12 set_property -dict { PACKAGE_PIN J15   IOSTANDARD LVCMOS33 } [get_ports { ce }];
13 set_property -dict { PACKAGE_PIN L16   IOSTANDARD LVCMOS33 } [get_ports { reset }];
14

29 ## LEDs
30 set_property -dict { PACKAGE_PIN H17   IOSTANDARD LVCMOS33 } [get_ports { count[0] }];
31 set_property -dict { PACKAGE_PIN K15   IOSTANDARD LVCMOS33 } [get_ports { count[1] }];
32 set_property -dict { PACKAGE_PIN J13   IOSTANDARD LVCMOS33 } [get_ports { count[2] }];
33 set_property -dict { PACKAGE_PIN N14   IOSTANDARD LVCMOS33 } [get_ports { count[3] }];
75
76 ##Buttons
77 set_property -dict { PACKAGE_PIN N17   IOSTANDARD LVCMOS33 } [get_ports { clk }];
```

Figura 12. Captura de pantalla las líneas del archivo de restricciones correspondientes a los puertos del circuito *dec7seg.vhd*.

En la Figura 13 se muestra el esquema de una FPGA con 8x8 CLB y con una línea dedicada de reloj que parte del pin resaltado en amarillo.

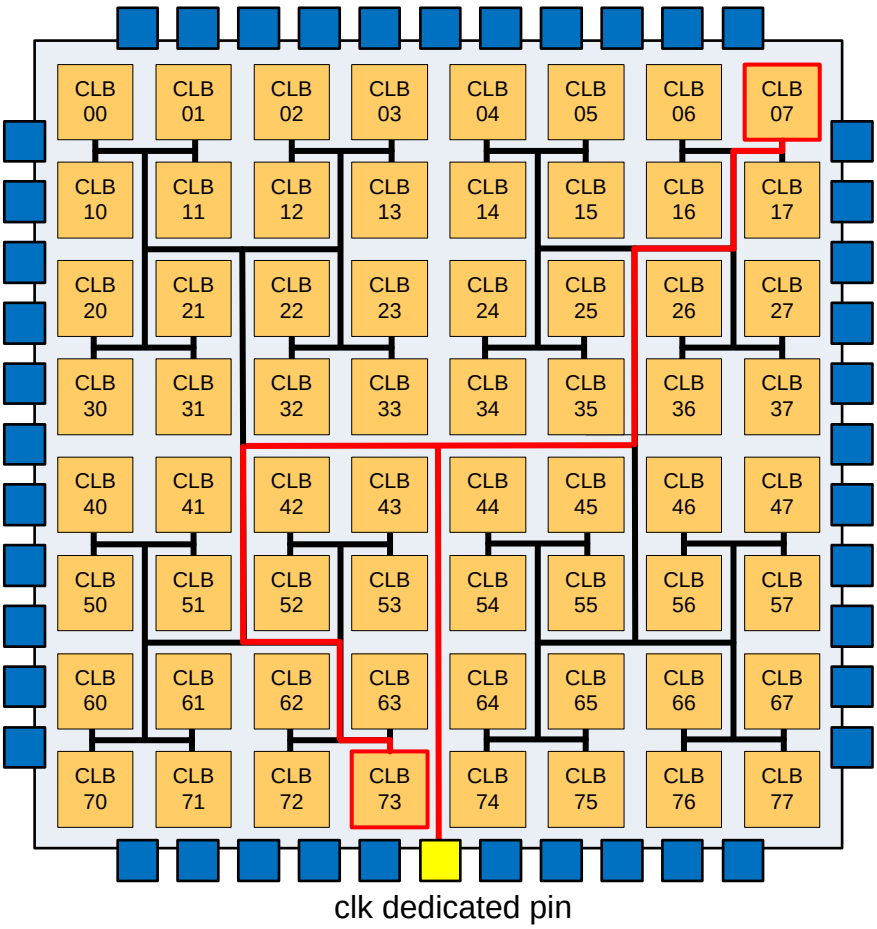

Figura 13. Esquema de una FPGA de 8x8 CLB con una línea de reloj.

El enrutamiento en el interior de la FPGA tiene la forma de lo que se conoce como un árbol en H (H-tree clock distribution). Esta distribución garantiza que la señal de reloj llega de manera simultánea a todos los CLB de la FPGA. Por ejemplo, el CLB73 se encuentra mucho más cerca de la entrada de reloj que el CLB07, y, sin embargo, la señal de reloj debe recorrer la misma distancia para alcanzar a ambos CLB. Esta condición es crítica en diseños muy exigentes en términos de velocidad.

En el caso del diseño del contador, se va a generar una señal de reloj pulsando en un botón de manera manual, así que la frecuencia de la señal generada será de pocos hertzios y, por tanto, no será crítica para el funcionamiento del circuito.

7.- Estructura interna de los CLB

En el apartado 2 de la Actividad 0 se ha descrito un esquema muy básico de FPGA, donde los CLB se usan para implementar puertas lógicas. En realidad, el CLB es un circuito más complejo en el que, además de realizar funciones combinacionales, se pueden realizar funciones secuenciales, es decir, con biestables. Esto ya se podía deducir de la Figura 13 del Apartado 5, en la que se muestra una señal de reloj que accede a cada CLB.

Los CLB son los recursos lógicos principales para implementar tanto circuitos combinacionales como secuenciales. Cada CLB está conectado a una matriz de conmutación para su acceso al sistema de interconexión general de la FPGA [1]. A su vez, cada CLB contiene 2 slices, como se muestra en la Figura 14.

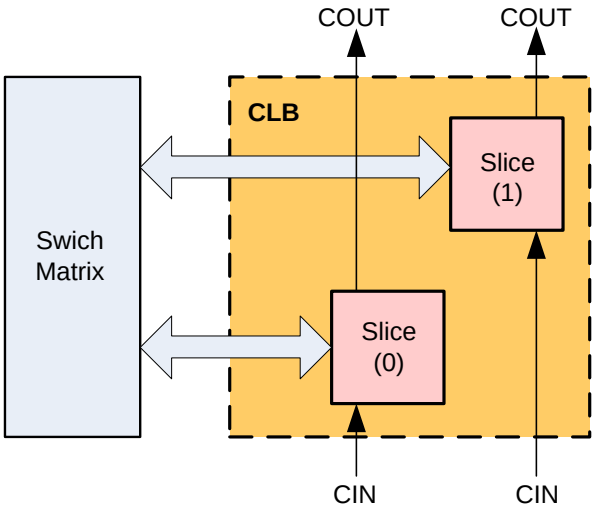

Figura 14. Disposición de los slices en el interior de un CLB.

En esta disposición las líneas CIN y COUT son líneas de acarreo dedicadas para poder realizar circuitos aritméticos eficientes empleando CLB vecinas a las que están conectadas directamente. En la Figura 15 se muestra esta conectividad en 4 CLB adyacentes de la esquina inferior izquierda de una FPGA.

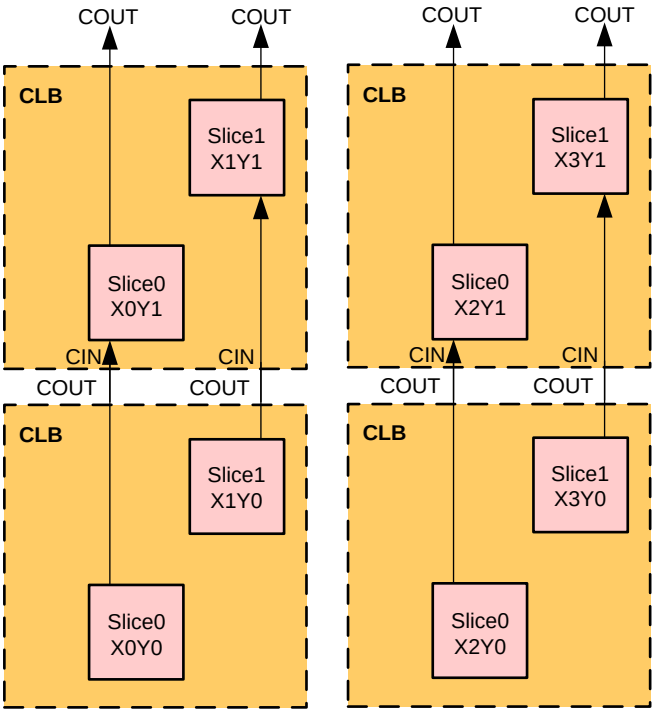

Figura 15. Relación de Filas y Columnas entre CLB y slice.

Las herramientas de Xilinx designan los slices siguiendo la siguiente nomenclatura:

- Una *X* seguida por un número identifica la posición de cada slice de una pareja, así como la posición deen columnas del slice. El número X cuenta slice comenzando desde la parte inferior de manera correlativa 0, 1 (la primera columna de CLB), 2, 3 (la segunda columna de CLB), etc.
- Una Y seguida por un número identifica una fila de slices. El número no cambia en un CLB, de tal manera que cuenta CLB conforme se asciende.

Una estructura interna simplificada de un slice (sin la lógica de acarreo y otros componentes) se muestra en la Figura 16. Se encuentran:

- 4 generadores de funciones lógicas (o look-up tables, LUT).
- 8 elementos de almacenamiento (flip-flops).
- Multiplexores de configuración.

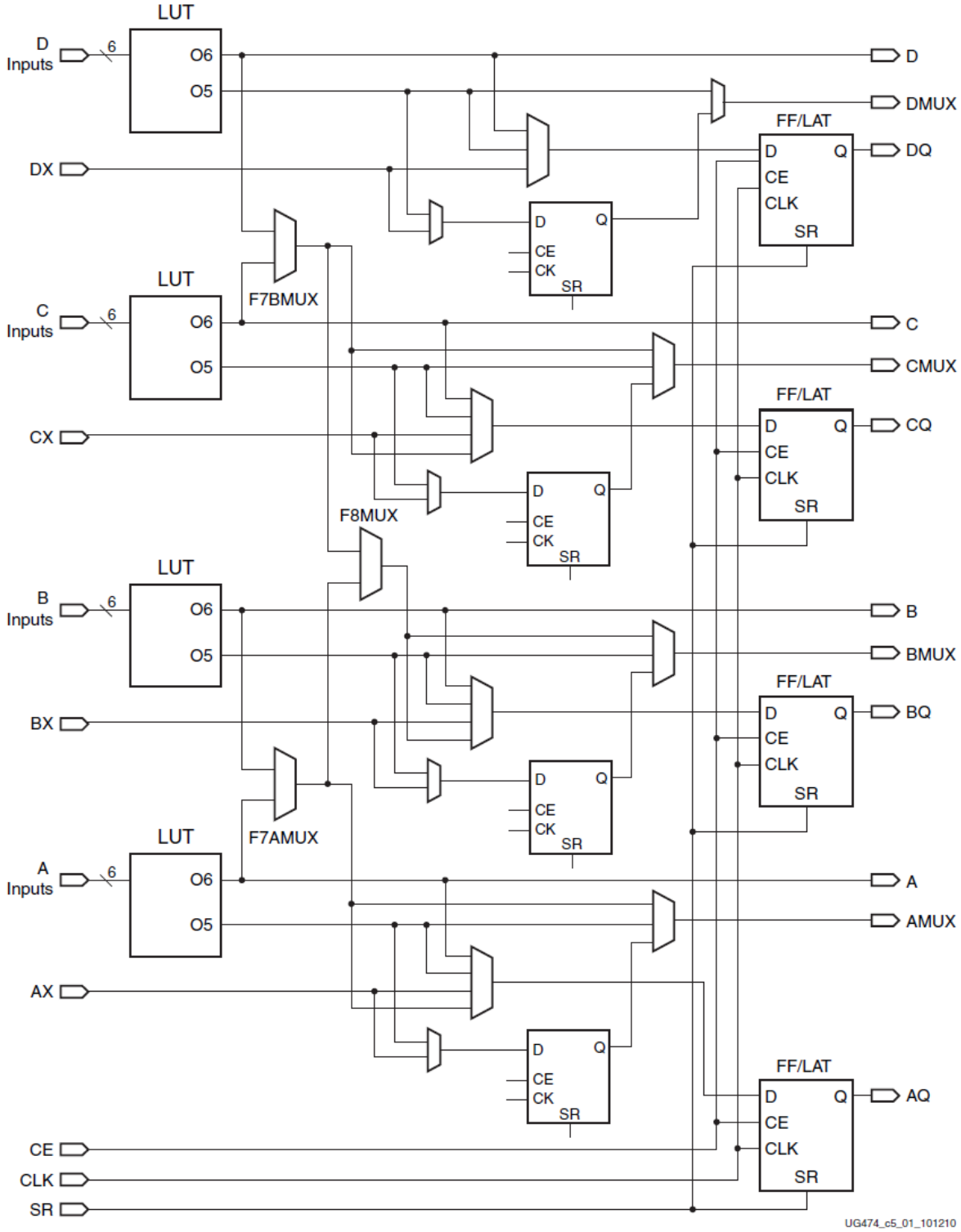

Figura 16. Slice simplificado de la serie 7 de FPGA de Xilinx [1].

Cada LUT tiene 6 entradas (LUT6). Este componente se usa para generar las funciones lógicas ya que se puede configurar como una memoria en la que se almacena la tabla de verdad de la función combinacional que se desea implementar [2].

A su vez, cada LUT6 se compone internamente de 2 LUT5, como se puede apreciar en la Figura 17 [2].

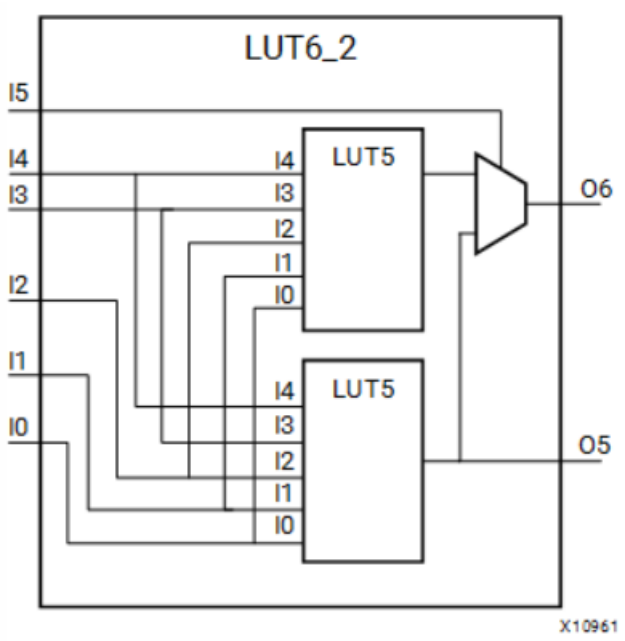

Figura 17. Esquema simplificado de una LUT6 [2].

Por tanto, cada LUT6 se puede usar para inferir:

- Cualquier función lógica de 6 variables.
- 2 funciones lógicas independientes cualesquiera de 5 variables (o menos).

Dentro de cada slice se encuentra 4 6LUT que, además se pueden combinar usando los multiplexores F7AMUX, F7BMUX y F8MUX. Esto permite inferir, en cada slice:

- 4 funciones lógicas de 6 variables.
- 2 funciones lógicas de 7 variables (una combinando LUT A y B con F7AMUX, y otra combinando LUT C y D con F7BMUX).
- 1 función lógica de 8 variables (combinando las 4 LUT con F7AMUX, F7BMUX y F8MUX).

Por último, es posible inferir funciones de un mayor número de variables combinando varios slices.

En la Tabla 2 se muestra una comparativa de las FPGA Artix-7 disponibles en la actualidad. En verde, se muestra la Artix-7 **XC7A100T-1CSG324C** y que consta de 15,850 slices (que incluyen 63,400 LUT6 y 126,800 flip-flop). Esta FPGA es la que está incluida en la tarjeta de desarrollo Nexys A7.

Dispositivo	Slices	6-input LUTs	Flip-Flops
7A12T	2,000	8,000	16,000
7A15T	2,600	10,400	20,800
7A25T	3,650	14,600	29,200
7A35T	5,200	20,800	41,600
7A50T	8,150	32,600	65,200
7A75T	11,800	47,200	94,400
7A100T	15,850	63,400	126,800
7A200T	33,650	134,600	269,200

Tabla 2. Recursos lógicos de las FPGA Artix-7 [1].

8.- Configuración del contador en la FPGA

El último paso consiste en configurar la tarjeta con el diseño del contador y haciendo uso de fichero de restricciones de usuario con los pines asignados a los puertos. Para realizar este paso hay que proceder de la misma manera que lo explicado en el Apartado 5 y el Apartado 6 de la Actividad 3. La Figura 18 muestra la tarjeta de desarrollo con el contador implementado en la FPGA Artix-7. **En este caso, el contador se encuentra habilitado (ce=1 y reset=0) y en la cuenta 7 (0111).**

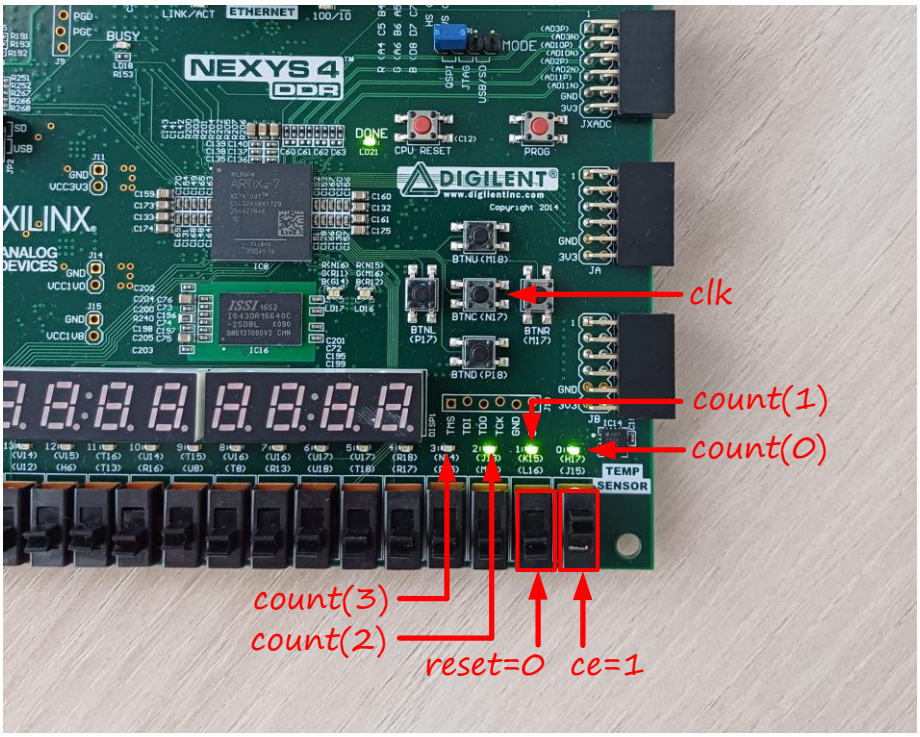

Figura 18. Foto de la tarjeta de desarrollo con el contador implementado en la FPGA.

9.- Recursos de la FPGA empleados

Es posible analizar los recursos que ha empleado el diseño de varias maneras.

La primera de ellas consiste en buscar la información en la pestaña de resumen del proyecto (*Project Summary*), que se encuentra por defecto en la ventana de trabajo (se puede abrir clicando en el icono Σ de la barra de herramientas de Vivado). Es posible ver la información de forma tabular y gráfica (ver Figuras 18 y 19).

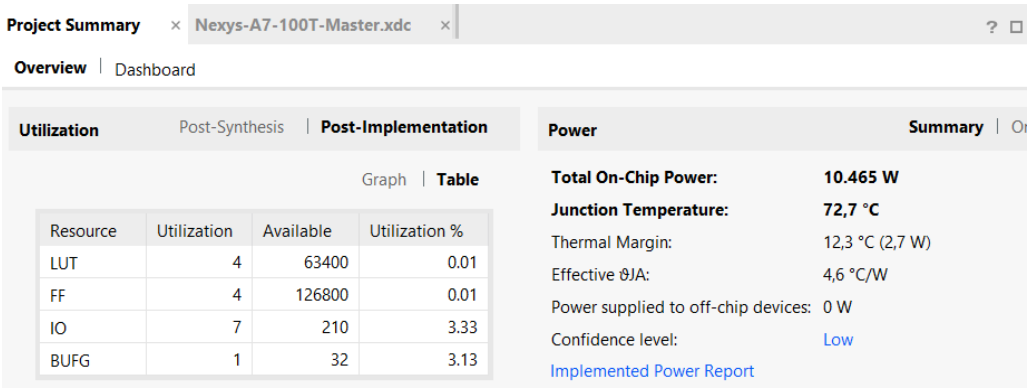

Figura 18. Captura de pantalla de la vista tabular de los recursos usados.

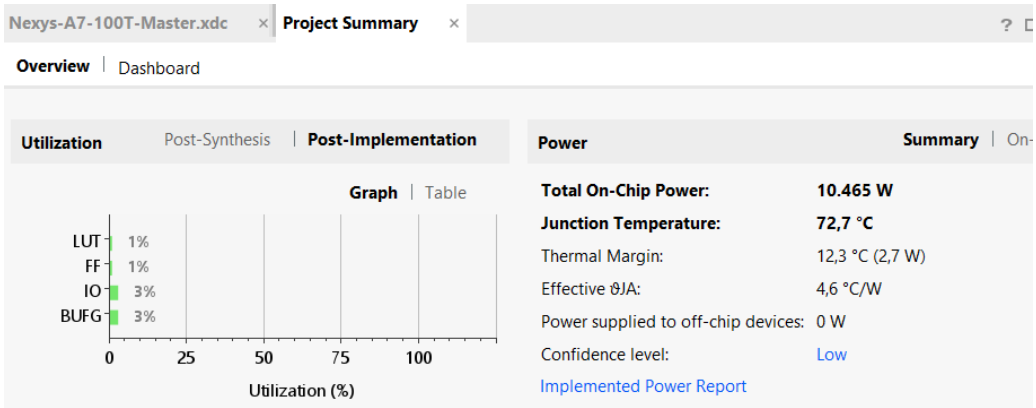

Figura 19. Captura de pantalla de la vista gráfica de los recursos usados.

En este sencillo diseño se puede contrastar los resultados obtenidos en el *report* con lo esperado. Analizando el circuito que se implementa (Figura 9) se puede apreciar que son necesarios 4 flip-flop (4 FF) y sus correspondientes 4 funciones de entrada para resolver el estado siguiente del circuito (4 LUT). Y, por otra parte, el circuito tiene 7 puertos de entrada/ salida: *ce*, *reset*, *clk* y los 4 de *count* (7 I/O).

La segunda manera de ver los recursos usados consiste en explorar en los informes que va generando Vivado en el proceso de implementación del circuito. Para acceder al informe adecuado debe buscarse en la ventana

inferior de Vivado, en la pestaña *Reports*, el informe de *Synthesis Utilization – Synth Design* (ver Figura 20).

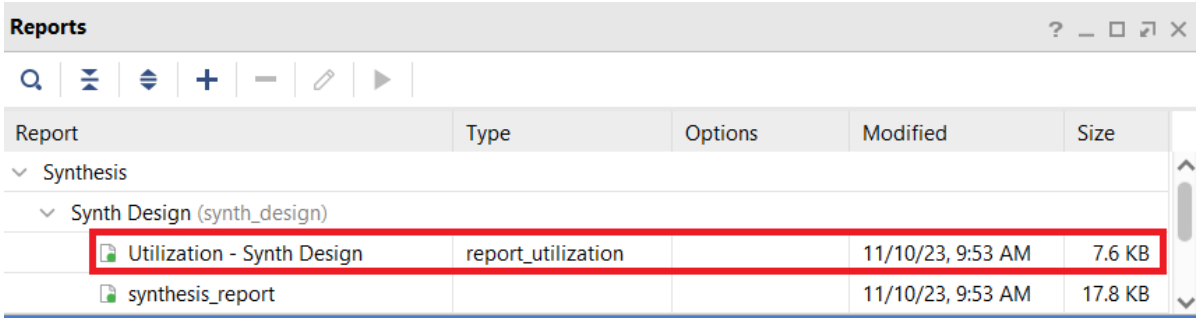

Figura 20. Captura de pantalla de la ventana *Reports* de Vivado.

En la Figura 21 se muestra la parte del informe concerniente a los recursos lógicos empleados.

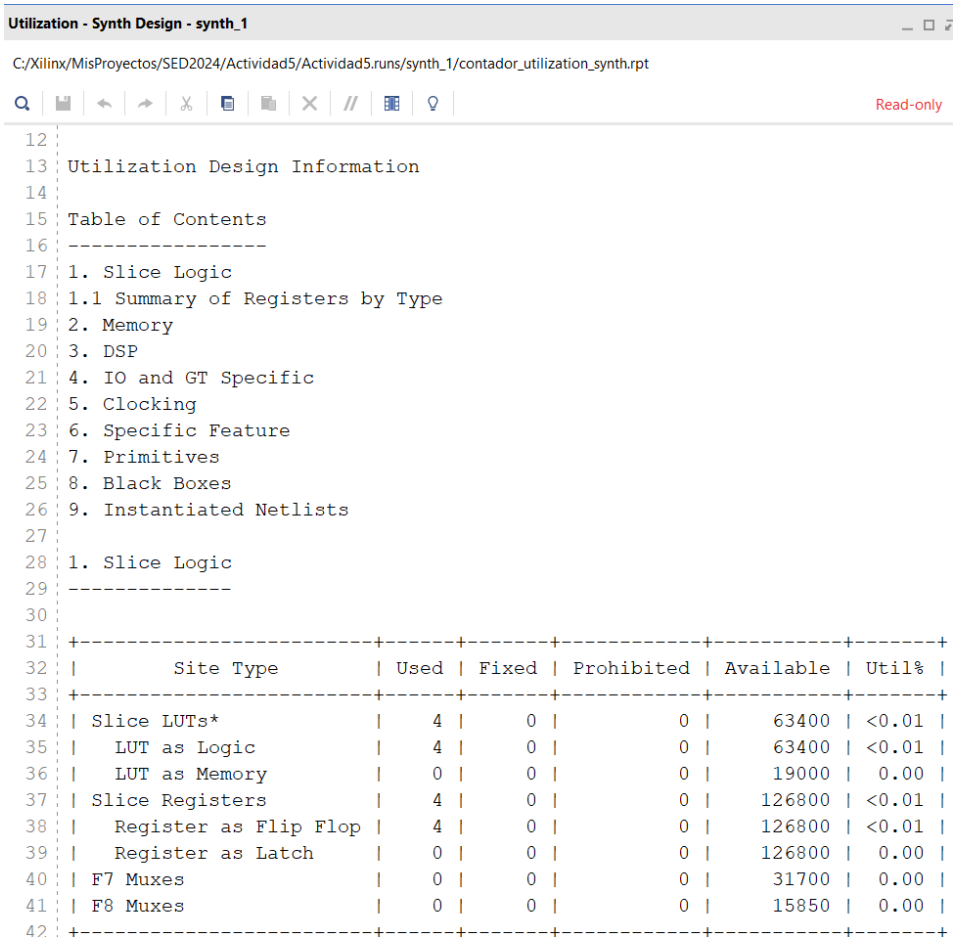

Figura 21. Captura de pantalla de la ventana del *report Utilization – Synth Design*.

Referencias

[1] https://docs.xilinx.com/v/u/en-US/ug474_7Series_CLB
[2] https://docs.xilinx.com/r/en-US/ug953-vivado-7series-libraries